CP Duel Backend Code + Deep Interview Q and A Guide

For every backend function or important code block: actual code first, deep explanation, execution flow, edge cases, production improvements, and interview questions with answers.

Project flow summary

Stage	Backend responsibility
1. Register/Login	Create or verify User, hash/compare password, issue JWT.
2. Create Room	Persist Room with host as first player and status waiting.
3. Join Room	Validate room, add second player, status becomes ready.
4. Socket Lobby	Players join Socket.IO room; lobby events are broadcast.
5. Start Duel	Host-only REST endpoint selects fair Codeforces problem and starts duel.
6. Poll Codeforces	Server polls submissions and verifies accepted solution after startedAt.
7. Finish Duel	Room/User records update, ratings change, gameResult is emitted.

Best high-level interview answer

The backend uses Express REST APIs for authenticated durable state changes, MongoDB/Mongoose for persistence, Socket.IO for real-time lobby and result updates, and Codeforces APIs for fair problem selection and server-side winner verification. The client never declares the winner; the backend verifies accepted submissions directly from Codeforces.

src/server.js

Application bootstrap

```
require('dotenv').config();
const express = require('express');
const http = require('http');
const cors = require('cors');
const connectDB = require('./config/db');
const { initSocket } = require('./sockets');

const authRoutes = require('./routes/auth');
const roomRoutes = require('./routes/rooms');
const duelRoutes = require('./routes/duel');

const app = express();
const server = http.createServer(app);

// Connect to MongoDB
connectDB();

// CORS - allow frontend origin
const allowedOrigins = [
  process.env.CLIENT_URL,
  'http://localhost:5173',
  'http://localhost:3000',
];

app.use(cors({
  origin: (origin, callback) => {
    // allow requests with no origin (mobile apps, curl, etc.)
    if (!origin) return callback(null, true);
    if (allowedOrigins.includes(origin)) return callback(null, true);
    return callback(new Error('CORS not allowed for: ' + origin));
  },
  credentials: true,
}));

app.use(express.json());

// Health check
app.get('/health', (req, res) => res.json({ status: 'ok' }));

// Routes
app.use('/api/auth', authRoutes);
app.use('/api/rooms', roomRoutes);
app.use('/api/duel', duelRoutes);

// Init Socket.IO (passes server + app for shared state)
initSocket(server);

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Deep explanation - what this code is responsible for

- This is the backend entry point. It composes infrastructure: env variables, Express app, HTTP server, MongoDB connection, CORS, JSON parser, REST routes, Socket.IO, and listen port.

- Business logic is intentionally not here. That keeps startup wiring separate from auth, room, duel, and Codeforces logic.

- The HTTP server wrapper is important because Socket.IO attaches to the raw HTTP server, while Express handles REST routes.

Execution flow

- Load .env.

- Create Express app.

- Create HTTP server.

- Connect DB.

- Register middleware.

- Mount routes.

- Initialize sockets.

- Start listening.

Edge cases / things interviewer may probe

- If MONGO_URI is bad, connectDB exits the process.

- If CLIENT_URL is missing, local origins still work but production frontend may fail CORS.

- CORS rejects unexpected browser origins.

How to improve in production

- Add centralized error middleware.

- Validate required env vars before starting.

- Add request logging and rate limiting.

- Use a stricter production CORS policy.

Possible interview questions and answers

Question	Answer
Why use http.createServer(app)?	Socket.IO needs the HTTP server instance so it can share the same network port with Express.
Why keep business logic out of server.js?	It keeps startup configuration clean and makes controllers/services easier to test.
What is the role of CORS here?	It controls which frontend origins are allowed to call the backend from a browser.
What would you add for production readiness?	Env validation, error middleware, request logs, rate limiting, and health checks with DB status.

src/config/db.js

connectDB()

```
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (err) {
    console.error('MongoDB connection error:', err.message);
    process.exit(1);
  }
};
```

Deep explanation - what this code is responsible for

- This function owns MongoDB connection setup through Mongoose.

- It reads MONGO_URI from environment variables, so secrets/config are not hardcoded.

- It exits the Node process if the database connection fails because almost every feature depends on MongoDB.

Execution flow

- Call mongoose.connect.

- Log connection host if successful.

- Catch error, log message, terminate process.

Edge cases / things interviewer may probe

- A bad URI, invalid credentials, or network issue will stop the app.

- There is no retry logic currently.

How to improve in production

- Add retry/backoff.

- Fail health checks if DB disconnects.

- Use connection event listeners for monitoring.

Possible interview questions and answers

Question	Answer
Why exit on DB failure?	The app cannot safely handle auth, rooms, or duel persistence without MongoDB.
Why use environment variable for MONGO_URI?	It keeps secrets out of source code and allows different DBs per environment.
What does Mongoose provide?	Schema modeling, validation, query helpers, middleware hooks, and ObjectId references.
How would you improve resilience?	Add retries and observability around connection events.

src/models/User.js

userSchema

```
const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true,
    trim: true,
    minlength: 3,
    maxlength: 20,
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true,
  },
  password: {
    type: String,
    required: true,
    minlength: 6,
  },
  codeforcesHandle: {
    type: String,
    required: true,
    trim: true,
  },
  rating: {
    type: Number,
    default: 1200,
  },
  wins: {
    type: Number,
    default: 0,
  },
  losses: {
    type: Number,
    default: 0,
  },
}, { timestamps: true });

// Hash password before save
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password
userSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

// Remove password from JSON responses
userSchema.methods.toJSON = function () {
  const obj = this.toObject();
  delete obj.password;
  return obj;
};
```

Deep explanation - what this code is responsible for

- The User schema defines account data and CP Duel-specific profile data.

- username/email uniqueness supports identity constraints.

- codeforcesHandle links your local user to Codeforces submissions.

- rating/wins/losses are game stats controlled by backend match results.

- timestamps are useful for auditing and debugging account lifecycle.

Execution flow

- Mongoose validates fields.

- Unique indexes help prevent duplicates.

- Defaults initialize rating and stats.

- Model methods/hooks below add security behavior.

Edge cases / things interviewer may probe

- unique is not a replacement for handling duplicate key errors.

- This schema does not validate whether Codeforces handle actually exists.

How to improve in production

- Verify Codeforces handle during registration.

- Add indexes explicitly for email/username.

- Normalize username/email consistently.

Possible interview questions and answers

Question	Answer
Why store codeforcesHandle?	The backend needs it to query Codeforces user.status for problem history and winner verification.
Why default rating to 1200?	It gives new users a neutral starting point for matchmaking and Elo calculations.
Is unique enough for validation?	No. It creates a DB constraint, but controllers should still handle duplicate key errors.
Why timestamps?	They help debug account creation and updates.

userSchema.pre('save')

```
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
}
```

Deep explanation - what this code is responsible for

- This Mongoose pre-save hook hashes passwords before storage.

- It keeps password security at the model layer, so any User.create or user.save path benefits automatically.

- isModified('password') avoids hashing an already-hashed password on unrelated updates.

Execution flow

- Check whether password changed.

- Generate bcrypt salt.

- Hash plaintext password.

- Replace password field.

- Call next().

Edge cases / things interviewer may probe

- findByIdAndUpdate does not trigger save middleware by default.

- If password updates are added later using update queries, they need separate hashing logic.

How to improve in production

- Add a dedicated password-change service.

- Use stronger password policy.

- Consider async error handling in the hook.

Possible interview questions and answers

Question	Answer
Why use bcrypt?	It is a slow password hashing algorithm designed to resist brute-force attacks.
Why check isModified?	Without it, saving a user would hash the existing hash again and break login.
Does this run for findByIdAndUpdate?	No, save middleware does not run for update queries by default.
Where should password hashing live?	Model layer is good here because it centralizes the guarantee.

comparePassword()

```
userSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};
```

Deep explanation - what this code is responsible for

- This instance method encapsulates password comparison for login.

- It accepts plaintext input and compares it to the stored bcrypt hash without decrypting anything.

Execution flow

- login() fetches user.

- login() calls user.comparePassword(password).

- bcrypt.compare returns true or false.

Edge cases / things interviewer may probe

- If user was fetched with password excluded, this would fail. Login fetches full user, so it works.

How to improve in production

- Keep password comparison timing and messages generic to avoid account enumeration.

Possible interview questions and answers

Question	Answer
Can bcrypt hashes be decrypted?	No. bcrypt verifies by hashing/checking the candidate, not by decrypting.
Why make this a model method?	It keeps authController cleaner and keeps password logic close to the schema.
What does it return?	A promise resolving to a boolean.
Why not compare strings directly?	The stored value is a salted hash, not plaintext.

toJSON()

```
userSchema.methods.toJSON = function () {
  const obj = this.toObject();
  delete obj.password;
  return obj;
};
```

Deep explanation - what this code is responsible for

- This customizes API serialization of User documents.

- It removes password from returned JSON even if controllers return the user object directly.

- It is a safety layer against leaking password hashes.

Execution flow

- Convert document to object.

- Delete password key.

- Return safe object.

Edge cases / things interviewer may probe

- It only affects JSON serialization, not raw database data.

- Other sensitive future fields would need removal too.

How to improve in production

- Use a serializer/DTO layer for larger apps.

- Also remove private fields like reset tokens if added.

Possible interview questions and answers

Question	Answer
Why remove password if it is hashed?	A password hash is still sensitive and should never be exposed.
When does toJSON run?	When Mongoose document is converted to JSON for responses.
Is this enough for all privacy?	No, future sensitive fields need explicit handling too.
Why is this useful in controllers?	Controllers can return user safely without manually deleting password every time.

src/models/Room.js

roomSchema

```
const roomSchema = new mongoose.Schema(
  {
    roomId: {
      type: String,
      default: () => nanoid(8).toUpperCase(),
      unique: true,
    },
    host: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    players: [
      {
        user: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
        username: String,
        codeforcesHandle: String,
        rating: Number,
      },
    ],
    status: {
      type: String,
      enum: ["waiting", "ready", "ongoing", "finished"],
      default: "waiting",
    },
    currentProblem: {
      contestId: Number,
      index: String,
      name: String,
      rating: Number,
      url: String,
      tags: [String],
    },
    winner: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      default: null,
    },
    winnerUsername: String,
    winnerSubmissionTime: Date,
    ratingDelta: {
      type: Number,
      default: null,
    },
    winnerNewRating: {
      type: Number,
      default: null,
    },
    loserNewRating: {
      type: Number,
      default: null,
    },
    startedAt: Date,
    finishedAt: Date,
  },
  { timestamps: true }
);
```

Deep explanation - what this code is responsible for

- Room is the central domain model for a duel.

- It stores lobby state, players, selected problem, winner, ratings, and timestamps.

- status acts as a state machine: waiting -> ready -> ongoing -> finished.

- players are embedded snapshots, which preserves username/handle/rating used at match time.

- currentProblem allows refresh/status pages to recover the active problem from DB.

Execution flow

- Room is created with host and one player.

- Second player joins and status becomes ready.

- startDuel writes currentProblem/status/startedAt.

- handleWinner writes winner/rating/finishedAt.

Edge cases / things interviewer may probe

- There is no schema-level max of 2 players; controller enforces it.

- There is no transaction around finish updates.

- roomId uniqueness could theoretically collide, though nanoid makes it unlikely.

How to improve in production

- Add indexes for roomId and players.user.

- Consider explicit state transition validation.

- Use transactions when finishing a duel.

Possible interview questions and answers

Question	Answer
Why is Room the main aggregate?	A duel's complete lifecycle is represented by one Room document.
Why embed players instead of only referencing users?	It stores a match-time snapshot of username, handle, and rating.
Why keep startedAt?	Polling uses it to ignore old accepted submissions.
Why store currentProblem?	So duel state survives refreshes and can be retrieved from MongoDB.

src/middleware/auth.js

authMiddleware()

```
const authMiddleware = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.userId).select('-password');
    if (!user) return res.status(401).json({ message: 'User not found' });
    req.user = user;
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }
};
```

Deep explanation - what this code is responsible for

- This middleware authenticates protected routes.

- It expects Authorization: Bearer .

- It verifies the JWT signature and expiration, then fetches the user from MongoDB.

- It attaches the authenticated user to req.user, so controllers do not trust user ids from the client.

Execution flow

- Read authorization header.

- Reject missing/non-Bearer token.

- Extract token.

- Verify with JWT_SECRET.

- Load user without password.

- Attach req.user.

- Call next().

Edge cases / things interviewer may probe

- Expired tokens return 401.

- Deleted users return 401.

- If JWT_SECRET changes, old tokens become invalid.

How to improve in production

- Add role/permission checks if needed.

- Use refresh tokens for longer sessions.

- Centralize error formatting.

Possible interview questions and answers

Question	Answer
Why fetch user after verifying token?	The token proves identity, but the DB fetch confirms the user still exists and gives fresh profile data.
Why not accept userId from req.body?	Clients can forge body values; JWT verification is trusted server-side authentication.
What does select('-password') do?	It excludes the password hash from req.user.
What status code is used for invalid auth?	401 Unauthorized.

src/routes/auth.js

Auth route mapping

```
router.post('/register', register);
router.post('/login', login);
router.get('/me', authMiddleware, getMe);
```

Deep explanation - what this code is responsible for

- This file maps auth URLs to controller functions.

- It keeps routing separate from implementation.

- Only /me is protected because register/login must be public.

Execution flow

- POST /register -> register.

- POST /login -> login.

- GET /me -> authMiddleware -> getMe.

Edge cases / things interviewer may probe

- If middleware order were wrong, /me would not have req.user.

How to improve in production

- Add request validation middleware before controllers.

Possible interview questions and answers

Question	Answer
Why keep route files thin?	It improves separation of concerns and readability.
Why is /me protected?	It returns the current user, which only exists after token verification.
Why are register/login public?	Users need them before they have a token.

src/routes/rooms.js

Room route mapping

```
router.post('/create', authMiddleware, createRoom);
router.post('/join', authMiddleware, joinRoom);
router.get('/history', authMiddleware, getHistory);  // add this -- must be before /:id
router.get('/:id', authMiddleware, getRoom);
```

Deep explanation - what this code is responsible for

- This file maps authenticated room endpoints.

- Every room operation uses authMiddleware because room actions depend on req.user.

- /history is placed before /:id to avoid Express treating 'history' as a room id.

Execution flow

- POST /create -> createRoom.

- POST /join -> joinRoom.

- GET /history -> getHistory.

- GET /:id -> getRoom.

Edge cases / things interviewer may probe

- Route ordering matters for dynamic params.

- Controllers still validate room state.

How to improve in production

- Add ownership/participant checks for reading private rooms if required.

Possible interview questions and answers

Question	Answer
Why must /history come before /:id?	Express matches routes in order; /:id would capture 'history'.
Why protect room routes?	The backend needs authenticated req.user to create/join/query user history.
Where is capacity checked?	joinRoom checks players.length >= 2.

src/routes/duel.js

Duel route mapping

```
router.post('/start', authMiddleware, startDuel);
router.get('/status/:roomId', authMiddleware, getDuelStatus);
```

Deep explanation - what this code is responsible for

- This maps duel start and status endpoints.

- Both are protected because duel state belongs to authenticated users.

- The controller performs host authorization and state checks.

Execution flow

- POST /start -> startDuel.

- GET /status/:roomId -> getDuelStatus.

Edge cases / things interviewer may probe

- Status endpoint currently does not verify room membership.

- Start endpoint must guard host-only behavior.

How to improve in production

- Add participant authorization for status if rooms should be private.

Possible interview questions and answers

Question	Answer
Why is start protected?	Only an authenticated host should start a duel.
Where is host-only logic?	Inside startDuel, comparing room.host with req.user._id.
Why have a status endpoint if sockets exist?	It supports refresh/reconnect and direct state recovery from DB.

src/controllers/authController.js

generateToken()

```
const generateToken = (userId) => {
  return jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn: '7d' });
};
```

Deep explanation - what this code is responsible for

- This helper creates the JWT returned after register/login.

- The token only stores userId, keeping payload minimal.

- expiresIn: '7d' controls session duration.

Execution flow

- Accept userId.

- Sign with JWT_SECRET.

- Return token string.

Edge cases / things interviewer may probe

- Weak or missing JWT_SECRET makes auth insecure.

- Token cannot be revoked unless secret changes or revocation logic exists.

How to improve in production

- Validate JWT_SECRET length.

- Consider refresh-token architecture and token revocation.

Possible interview questions and answers

Question	Answer
What is inside the JWT?	Only userId in this project.
Why not store password/email in token?	Smaller payload and less sensitive data exposure.
What happens after 7 days?	jwt.verify rejects it and middleware returns 401.

register()

```
const register = async (req, res) => {
  try {
    const { username, email, password, codeforcesHandle } = req.body;

    if (!username || !email || !password || !codeforcesHandle) {
      return res.status(400).json({ message: 'All fields are required' });
    }

    // Check existing user
    const existing = await User.findOne({ $or: [{ email }, { username }] });
    if (existing) {
      return res.status(409).json({ message: 'Username or email already taken' });
    }

    const user = await User.create({ username, email, password, codeforcesHandle });
    const token = generateToken(user._id);

    res.status(201).json({ token, user });
  } catch (err) {
    console.error('Register error:', err);
    res.status(500).json({ message: 'Server error during registration' });
  }
};
```

Deep explanation - what this code is responsible for

- register creates a new user account and immediately returns a token.

- It validates required fields, checks duplicates, creates the user, and relies on User model hook for password hashing.

- Returning token after registration creates a smooth login-after-signup flow.

Execution flow

- Read body.

- Validate fields.

- Check existing email/username.

- Create user.

- Generate token.

- Return 201.

Edge cases / things interviewer may probe

- Duplicate key race can still happen between check and create.

- Codeforces handle is not verified here.

- Password policy is basic.

How to improve in production

- Catch Mongo duplicate key errors.

- Validate email format and handle existence.

- Add stronger password rules.

Possible interview questions and answers

Question	Answer
Where is the password hashed?	In User.js pre-save middleware triggered by User.create.
Why check both email and username?	Both are unique identifiers and should not collide.
Why return 409 for duplicate?	409 Conflict communicates resource uniqueness conflict.
What would you improve?	Handle DB duplicate errors and verify Codeforces handle.

login()

```
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: 'Email and password are required' });
    }

    const user = await User.findOne({ email });
    if (!user) return res.status(401).json({ message: 'Invalid credentials' });

    const isMatch = await user.comparePassword(password);
    if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

    const token = generateToken(user._id);
    res.json({ token, user });
  } catch (err) {
    console.error('Login error:', err);
    res.status(500).json({ message: 'Server error during login' });
  }
};
```

Deep explanation - what this code is responsible for

- login authenticates an existing user.

- It checks required credentials, finds user by email, compares password using bcrypt, and returns a token.

- It uses a generic invalid credentials response for missing user or wrong password.

Execution flow

- Read email/password.

- Validate present.

- Find user.

- Compare password.

- Generate token.

- Return user/token.

Edge cases / things interviewer may probe

- No rate limiting means brute-force attempts are possible.

- Email normalization depends on schema lowercase behavior.

How to improve in production

- Add login rate limiting.

- Add account lockout or CAPTCHA for repeated failures.

- Add audit logs.

Possible interview questions and answers

Question	Answer
Why use generic 'Invalid credentials'?	To avoid revealing whether an email exists.
Why does login need full user password?	comparePassword needs the stored hash.
What prevents plaintext storage?	User pre-save hook hashes on create.
What security feature is missing?	Rate limiting on login attempts.

getMe()

```
const getMe = async (req, res) => {
  // req.user is set by authMiddleware
  res.json({ user: req.user });
};
```

Deep explanation - what this code is responsible for

- getMe returns authenticated user profile from req.user.

- It is intentionally simple because authMiddleware already verified token and loaded user.

- Frontend can use it to restore session state after refresh.

Execution flow

- authMiddleware runs.

- req.user is attached.

- getMe returns { user: req.user }.

Edge cases / things interviewer may probe

- If middleware is skipped, req.user would be undefined.

How to improve in production

- Add standardized response shape if API grows.

Possible interview questions and answers

Question	Answer
Why is getMe useful?	It lets the frontend check the current token and load user data.
Where does req.user come from?	authMiddleware.
Does it expose password?	No, middleware selects user without password.

src/controllers/roomController.js

createRoom()

```
const createRoom = async (req, res) => {
  try {
    const user = req.user;

    const room = await Room.create({
      host: user._id,
      players: [{
        user: user._id,
        username: user.username,
        codeforcesHandle: user.codeforcesHandle,
        rating: user.rating,
      }],
      status: 'waiting',
    });

    res.status(201).json({ room });
  } catch (err) {
    console.error('createRoom error:', err);
    res.status(500).json({ message: 'Failed to create room' });
  }
};
```

Deep explanation - what this code is responsible for

- createRoom starts a lobby.

- The authenticated user becomes both host and first player.

- The Room schema generates the public roomId.

- The room starts with status waiting because it needs a second player.

Execution flow

- Read req.user.

- Create Room with host/player snapshot.

- Return 201 with room.

Edge cases / things interviewer may probe

- A user can create multiple waiting rooms.

- No cleanup for abandoned rooms.

How to improve in production

- Limit active rooms per user.

- Expire old waiting rooms.

- Use indexes for host/status.

Possible interview questions and answers

Question	Answer
Why embed host as first player?	The host participates in the duel, not just owns the room.
Why status waiting?	Only one player is present.
Where is roomId generated?	Room schema default using nanoid.
What improvement would you suggest?	Prevent users from creating unlimited stale rooms.

joinRoom()

```
const joinRoom = async (req, res) => {
  try {
    const { roomId } = req.body;
    const user = req.user;

    if (!roomId) return res.status(400).json({ message: 'roomId is required' });

    const room = await Room.findOne({ roomId });
    if (!room) return res.status(404).json({ message: 'Room not found' });
    if (room.status !== 'waiting') return res.status(400).json({ message: 'Room is not open for joining'
     });
    if (room.players.length >= 2) return res.status(400).json({ message: 'Room is full' });

    // Check if user is already in room
    const alreadyIn = room.players.some((p) => String(p.user) === String(user._id));
    if (alreadyIn) return res.json({ room }); // idempotent

    room.players.push({
      user: user._id,
      username: user.username,
      codeforcesHandle: user.codeforcesHandle,
      rating: user.rating,
    });
    room.status = 'ready';
    await room.save();

    res.json({ room });
  } catch (err) {
    console.error('joinRoom error:', err);
    res.status(500).json({ message: 'Failed to join room' });
  }
};
```

Deep explanation - what this code is responsible for

- joinRoom moves a lobby from waiting to ready.

- It validates room existence, status, capacity, and duplicate membership.

- It is idempotent if the same user tries to join again, returning the room without duplicate insertion.

Execution flow

- Read roomId.

- Find Room.

- Reject missing/not waiting/full.

- Check alreadyIn.

- Push second player.

- Set ready.

- Save.

Edge cases / things interviewer may probe

- Concurrent joins could race and overfill without atomic update.

- The host can trigger alreadyIn behavior if rejoining.

How to improve in production

- Use atomic conditional update for capacity.

- Add transaction or unique player constraint per room.

Possible interview questions and answers

Question	Answer
Why check alreadyIn?	To avoid duplicate player entries on retry/refresh.
Why set status ready?	Two players are present and host can start.
Can two users join at the same time?	There is a possible race; atomic updates would be safer.
Why reject non-waiting rooms?	Finished/ongoing/ready rooms should not accept new players.

getRoom()

```
const getRoom = async (req, res) => {
  try {
    const room = await Room.findOne({ roomId: req.params.id });
    if (!room) return res.status(404).json({ message: 'Room not found' });
    res.json({ room });
  } catch (err) {
    console.error('getRoom error:', err);
    res.status(500).json({ message: 'Failed to get room' });
  }
};
```

Deep explanation - what this code is responsible for

- getRoom returns persisted room state by public roomId.

- It helps frontend recover after refresh or direct navigation.

- It returns 404 if the room code is invalid.

Execution flow

- Read req.params.id.

- Find room by roomId.

- Return room or 404.

Edge cases / things interviewer may probe

- Currently any authenticated user can fetch any room if they know roomId.

How to improve in production

- Check that req.user is a participant if rooms should be private.

Possible interview questions and answers

Question	Answer
Why fetch by roomId instead of _id?	roomId is the shareable code users type.
Why is this needed with sockets?	Sockets do not help after refresh; DB read restores state.
What auth check might be added?	Verify current user is in room.players.

getHistory()

```
const getHistory = async (req, res) => {
  try {
    const userId = req.user._id;

    const rooms = await Room.find({
      status: 'finished',
      'players.user': userId,
    })
      .sort({ finishedAt: -1 })
      .limit(20);

    const history = rooms.map((room) => {
      const me       = room.players.find(p => String(p.user) === String(userId));
      const opponent = room.players.find(p => String(p.user) !== String(userId));
      const won      = String(room.winner) === String(userId);

      return {
        roomId:          room.roomId,
        problem:         room.currentProblem,
        outcome:         won ? 'win' : 'loss',
        opponent:        opponent?.username || 'Unknown',
        opponentHandle:  opponent?.codeforcesHandle || '',
        finishedAt:      room.finishedAt,
        ratingDelta:     won ? room.ratingDelta : -(room.ratingDelta || 0),
        myRating:        won ? room.winnerNewRating : room.loserNewRating,
      };
    });

    res.json({ history });
  } catch (err) {
    console.error('getHistory error:', err);
    res.status(500).json({ message: 'Failed to fetch history' });
  }
};
```

Deep explanation - what this code is responsible for

- getHistory derives match history from finished Room documents.

- It filters rooms where current user participated, sorts newest first, and limits results.

- It maps raw Room documents into UI-friendly summaries including outcome and rating delta.

Execution flow

- Read req.user._id.

- Query finished rooms containing user.

- Sort/limit.

- Find opponent.

- Compute won.

- Return history array.

Edge cases / things interviewer may probe

- If room had malformed player data, opponent may be Unknown.

- It depends on rating fields being correctly written by handleWinner.

How to improve in production

- Paginate history.

- Create indexes on status and players.user.

- Store match history in separate collection for analytics if needed.

Possible interview questions and answers

Question	Answer
Why derive history from Room?	Finished rooms already contain problem, players, winner, and rating result.
Why ratingDelta negative for losses?	It makes the user's perspective clear in history.
Why limit to 20?	To keep response small.
What index would help?	An index on status and players.user.

src/controllers/duelController.js

setIO()

```
let _io = null;
const setIO = (io) => { _io = io; };
```

Deep explanation - what this code is responsible for

- setIO stores the Socket.IO server instance for controller use.

- It is simple dependency injection: sockets/index.js creates io and gives it to duelController.

- This lets startDuel emit duelStarted from inside a REST request handler.

Execution flow

- initSocket creates io.

- initSocket calls setIO(io).

- startDuel later uses _io.to(roomId).emit(...).

Edge cases / things interviewer may probe

- Module-level state is simple but not ideal for complex tests or multi-process setups.

How to improve in production

- Pass io through app context or use a small event bus in larger systems.

Possible interview questions and answers

Question	Answer
Why does a controller need io?	Starting a duel via REST must notify connected clients in real time.
What pattern is this?	A lightweight dependency injection/shared module reference pattern.
Is this scalable across multiple server instances?	Not by itself; multi-instance Socket.IO needs an adapter like Redis.

startDuel()

```
const startDuel = async (req, res) => {
  try {
    const { roomId } = req.body;
    const user = req.user;

    if (!roomId) return res.status(400).json({ message: 'roomId required' });

    const room = await Room.findOne({ roomId });
    if (!room) return res.status(404).json({ message: 'Room not found' });

    // Only the host can start
    if (String(room.host) !== String(user._id)) {
      return res.status(403).json({ message: 'Only the host can start the duel' });
    }
    if (room.players.length < 2) {
      return res.status(400).json({ message: 'Need 2 players to start' });
    }
    if (room.status === 'ongoing') {
      return res.status(400).json({ message: 'Duel already in progress' });
    }

    // Calculate average rating of the two players
    const avgRating = Math.round(
      room.players.reduce((sum, p) => sum + (p.rating || 1200), 0) / room.players.length
    );

    // Select a Codeforces problem
    const [p1, p2] = room.players;
    const problem = await selectProblem(p1.codeforcesHandle, p2.codeforcesHandle, avgRating);

    const startedAt = new Date();
    room.currentProblem = problem;
    room.status = 'ongoing';
    room.startedAt = startedAt;
    await room.save();

    // Notify all players in socket room
    if (_io) {
      _io.to(roomId).emit('duelStarted', { problem, startedAt });
    }

    // Begin polling for submissions
    const playersForPolling = room.players.map((p) => ({
      userId: p.user,
      username: p.username,
      codeforcesHandle: p.codeforcesHandle,
    }));
    startPolling(roomId, problem, playersForPolling, _io, startedAt);

    res.json({ problem, startedAt });
  } catch (err) {
    console.error('startDuel error:', err);
    res.status(500).json({ message: err.message || 'Failed to start duel' });
  }
};
```

Deep explanation - what this code is responsible for

- startDuel is the main orchestration function for a match.

- It protects game integrity by checking host authorization and room state before selecting a problem.

- It calculates average rating, delegates problem selection to Codeforces service, persists the duel start, emits real-time event, and starts server-side polling.

Execution flow

- Validate roomId.

- Load room.

- Check current user is host.

- Check 2 players.

- Check not ongoing.

- Compute avgRating.

- selectProblem.

- Save currentProblem/status/startedAt.

- Emit duelStarted.

- startPolling.

- Return response.

Edge cases / things interviewer may probe

- If Codeforces API fails, duel start fails.

- If _io is null, HTTP still returns but clients may not get socket event.

- It permits starting a finished room again unless status logic is tightened.

How to improve in production

- Disallow start unless status is ready.

- Use transactions/state transition guards.

- Cache Codeforces problemset.

- Add better errors for Codeforces failures.

Possible interview questions and answers

Question	Answer
Why only host can start?	It prevents the opponent from starting before both players are ready.
Why calculate average rating?	To select a problem around both players' skill level.
Why store startedAt?	Polling ignores submissions before duel start.
Why both REST and socket emit?	REST performs the state change; socket informs both clients live.
What is a weakness?	No strict status === ready check and no transaction around state transition.

getDuelStatus()

```
const getDuelStatus = async (req, res) => {
  try {
    const room = await Room.findOne({ roomId: req.params.roomId });
    if (!room) return res.status(404).json({ message: 'Room not found' });

    res.json({
      status: room.status,
      currentProblem: room.currentProblem,
      winner: room.winnerUsername,
      winnerSubmissionTime: room.winnerSubmissionTime,
      startedAt: room.startedAt,
      finishedAt: room.finishedAt,
    });
  } catch (err) {
    console.error('getDuelStatus error:', err);
    res.status(500).json({ message: 'Failed to get duel status' });
  }
};
```

Deep explanation - what this code is responsible for

- getDuelStatus returns a compact view of current duel state.

- It is useful after refresh, reconnect, or result page navigation.

- It does not return every Room field, only what duel UI needs.

Execution flow

- Read roomId param.

- Find Room.

- Return status/problem/winner/timestamps or 404.

Edge cases / things interviewer may probe

- Currently any authenticated user with roomId can read status.

How to improve in production

- Add participant check if privacy is required.

Possible interview questions and answers

Question	Answer
Why have this endpoint?	Sockets are not persistent across refreshes; this endpoint restores state.
Why return currentProblem?	Frontend needs problem link/details after refresh.
Should it check membership?	For private rooms, yes.

src/services/codeforcesService.js

fetchProblems()

```
const fetchProblems = async () => {
  const res = await axios.get(`${CF_BASE}/problemset.problems`);
  if (res.data.status !== 'OK') throw new Error('Codeforces API error');
  return res.data.result.problems;
};
```

Deep explanation - what this code is responsible for

- fetchProblems wraps Codeforces problemset.problems.

- It validates Codeforces API status and returns only the problems array.

- This keeps external API details out of the controller.

Execution flow

- GET /problemset.problems.

- Check status OK.

- Return result.problems.

Edge cases / things interviewer may probe

- Large response could be slow.

- No caching means every duel start fetches all problems.

How to improve in production

- Cache problemset in memory with TTL.

- Add retry/backoff and timeout.

Possible interview questions and answers

Question	Answer
Why wrap the API call?	It isolates Codeforces integration in a service layer.
Why check status OK?	Codeforces may respond with errors even if HTTP request succeeds.
What is the main performance issue?	Fetching entire problemset repeatedly.

fetchSolvedProblems()

```
const fetchSolvedProblems = async (handle) => {
  const res = await axios.get(`${CF_BASE}/user.status`, {
    params: { handle, from: 1, count: 1000 },
  });
  if (res.data.status !== 'OK') throw new Error(`CF API error for handle: ${handle}`);

  const solved = new Set();
  for (const sub of res.data.result) {
    if (sub.verdict === 'OK' && sub.problem) {
      solved.add(`${sub.problem.contestId}-${sub.problem.index}`);
    }
  }
  return solved;
};
```

Deep explanation - what this code is responsible for

- fetchSolvedProblems builds a Set of accepted problems for one handle.

- The Set allows fast membership checks when filtering candidates.

- It uses recent 1000 submissions, which is practical but not a perfect full solved archive.

Execution flow

- GET /user.status.

- Check status OK.

- Loop submissions.

- If verdict OK, add contestId-index key to Set.

- Return Set.

Edge cases / things interviewer may probe

- Only last 1000 submissions are considered.

- Invalid Codeforces handle causes API error.

- Repeated accepted submissions collapse into one Set entry.

How to improve in production

- Validate handle at registration.

- Increase/cycle pagination if full solve history is needed.

- Cache solved sets briefly.

Possible interview questions and answers

Question	Answer
Why use a Set?	O(1) lookup when checking if a candidate problem is solved.
Why key as contestId-index?	That uniquely identifies a Codeforces problem in this app.
Is count 1000 perfect?	No, it is a practical approximation.
What verdict counts?	Only OK, meaning accepted.

selectProblem()

```
const selectProblem = async (handle1, handle2, avgRating) => {
  const lo = avgRating - 200;
  const hi = avgRating + 200;

  // Banned tags (special or non-standard problems)
  const BANNED_TAGS = ['*special', 'interactive'];

  const [problems, solved1, solved2] = await Promise.all([
    fetchProblems(),
    fetchSolvedProblems(handle1),
    fetchSolvedProblems(handle2),
  ]);

  // Filter suitable problems
  const candidates = problems.filter((p) => {
    const key = `${p.contestId}-${p.index}`;
    const inRatingRange = p.rating >= lo && p.rating <= hi;
    const notSolvedByEither = !solved1.has(key) && !solved2.has(key);
    const noGym = p.contestId < 100000; // Gym contests have IDs >= 100000
    const noIndex = p.contestId !== undefined && p.index !== undefined;
    const hasTags = p.tags && !p.tags.some((t) => BANNED_TAGS.includes(t));
    return inRatingRange && notSolvedByEither && noGym && noIndex && hasTags;
  });

  if (candidates.length === 0) {
    throw new Error('No suitable problem found in rating range');
  }

  const pick = candidates[Math.floor(Math.random() * candidates.length)];
  return {
    contestId: pick.contestId,
    index: pick.index,
    name: pick.name,
    rating: pick.rating,
    tags: pick.tags || [],
    url: `https://codeforces.com/problemset/problem/${pick.contestId}/${pick.index}`,
  };
};
```

Deep explanation - what this code is responsible for

- selectProblem is the problem matching algorithm.

- It chooses a problem around average rating that neither player has solved.

- It filters out Gym, special, and interactive problems to keep the duel standard and fair.

- It fetches problemset and solved data concurrently using Promise.all.

Execution flow

- Compute rating range.

- Define banned tags.

- Fetch problems/solved sets in parallel.

- Filter candidates.

- Throw if none.

- Pick random candidate.

- Return normalized problem object with URL.

Edge cases / things interviewer may probe

- Random selection means difficulty can still vary within range.

- No caching may make start slower.

- If no problem exists, duel cannot start.

How to improve in production

- Cache problemset.

- Add fallback wider rating range.

- Avoid recently used problems.

- Improve fairness with tag preferences.

Possible interview questions and answers

Question	Answer
Why use avgRating +/- 200?	It gives a skill-appropriate window without being too narrow.
Why use Promise.all?	Independent API calls run concurrently, reducing wait time.
Why ban interactive problems?	They are not suitable for simple first-AC duel format.
What happens if no candidate exists?	It throws 'No suitable problem found in rating range'.
How would you improve selection?	Cache data, widen fallback range, and avoid repeats.

getRecentSubmissions()

```
const getRecentSubmissions = async (handle, count = 10) => {
  const res = await axios.get(`${CF_BASE}/user.status`, {
    params: { handle, from: 1, count },
  });
  if (res.data.status !== 'OK') return [];
  return res.data.result;
};
```

Deep explanation - what this code is responsible for

- getRecentSubmissions supports active polling.

- It fetches a small number of recent submissions for a Codeforces handle.

- It returns [] on non-OK status, allowing polling to continue instead of crashing.

Execution flow

- GET /user.status with handle/from/count.

- If status not OK return [].

- Return result array.

Edge cases / things interviewer may probe

- Network errors from axios still throw and are caught by pollingService.

- count must be enough to catch submissions between polls.

How to improve in production

- Set request timeout.

- Add retry/backoff.

- Log non-OK statuses with context.

Possible interview questions and answers

Question	Answer
Why count only recent submissions?	During a duel, only new submissions matter.
Why return [] for non-OK?	It lets polling keep running and try again later.
Where is it used?	startPolling checks both players' recent submissions.
What risk exists?	Codeforces API delays/rate limits can delay winner detection.

src/services/pollingService.js

activePolls

```
const activePolls = new Map();
```

Deep explanation - what this code is responsible for

- activePolls stores one interval per roomId.

- It prevents duplicate polling loops for the same room.

- It is runtime memory, not persistent state.

Execution flow

- startPolling checks activePolls.

- stopPolling clears and deletes an entry.

Edge cases / things interviewer may probe

- Server restart loses all active polls.

- Multiple backend instances would each have separate maps.

How to improve in production

- Use a job queue or database-backed scheduler for production.

- Use Redis if scaling Socket.IO/multiple workers.

Possible interview questions and answers

Question	Answer
Why need activePolls?	To avoid multiple intervals calling Codeforces for the same room.
What is the limitation?	It disappears on server restart.
How to improve?	Use persistent background jobs or queue workers.

startPolling()

```
const startPolling = (roomId, problem, players, io, startedAt) => {
  if (activePolls.has(roomId)) {
    console.log(`Poll already running for room ${roomId}`);
    return;
  }

  const POLL_INTERVAL = 6000; // 6 seconds

  const intervalId = setInterval(async () => {
    try {
      // Check each player's recent submissions in parallel
      const checks = players.map(async (player) => {
        const subs = await getRecentSubmissions(player.codeforcesHandle, 15);
        for (const sub of subs) {
          // Only look at submissions after the duel started
          const subTime = new Date(sub.creationTimeSeconds * 1000);
          if (subTime < startedAt) continue;

          if (
            sub.verdict === "OK" &&
            sub.problem.contestId === problem.contestId &&
            sub.problem.index === problem.index
          ) {
            return { player, submission: sub, subTime };
          }
        }
        return null;
      });

      const results = await Promise.all(checks);
      const winner = results.find((r) => r !== null);

      if (winner) {
        stopPolling(roomId);
        await handleWinner(roomId, winner, io);
      }
    } catch (err) {
      console.error(`Polling error for room ${roomId}:`, err.message);
    }
  }, POLL_INTERVAL);

  activePolls.set(roomId, intervalId);
  console.log(`Started polling for room ${roomId}`);
};
```

Deep explanation - what this code is responsible for

- startPolling is the winner-detection loop.

- It checks Codeforces every 6 seconds for both players.

- It only accepts submissions after startedAt and for the exact selected contestId/index.

- The client never decides the winner; the server verifies through Codeforces.

Execution flow

- Guard duplicate poll.

- Create 6s interval.

- Fetch both players' submissions concurrently.

- Ignore old submissions.

- Find OK verdict for exact problem.

- If found, stop polling and handle winner.

Edge cases / things interviewer may probe

- If both solve in same interval, results.find picks first non-null by players array order, not necessarily earliest subTime.

- Polling continues forever if nobody solves and no timeout exists.

- Server restart stops polling.

How to improve in production

- Compare all winner candidates by earliest subTime.

- Add max duel duration/timeout.

- Persist polling jobs.

- Handle Codeforces rate limits.

Possible interview questions and answers

Question	Answer
Why filter by startedAt?	To prevent an old accepted submission from winning.
Why poll instead of client submit?	The source of truth is Codeforces; server verification prevents cheating.
What if both solve simultaneously?	Current code may pick player order; better code should compare submission times.
Why every 6 seconds?	It balances responsiveness with API load.
What is a production weakness?	In-memory interval disappears on restart.

stopPolling()

```
const stopPolling = (roomId) => {
  if (activePolls.has(roomId)) {
    clearInterval(activePolls.get(roomId));
    activePolls.delete(roomId);
    console.log(`Stopped polling for room ${roomId}`);
  }
};
```

Deep explanation - what this code is responsible for

- stopPolling stops Codeforces checks for a room.

- It clears the interval and removes roomId from activePolls.

- This avoids unnecessary API calls after a winner is found.

Execution flow

- Check map.

- clearInterval.

- delete map entry.

- Log stop.

Edge cases / things interviewer may probe

- No-op if roomId is missing, which is safe.

How to improve in production

- Call it on explicit cancellation/timeout too, not only winner.

Possible interview questions and answers

Question	Answer
Why clear the interval?	Otherwise the backend would keep polling forever.
What happens if roomId is not in map?	Nothing; function safely no-ops.
Where is it called?	Inside startPolling when a winner is detected.

calcEloDelta()

```
const calcEloDelta = (winnerRating, loserRating) => {
  const K = 32;
  const expected = 1 / (1 + Math.pow(10, (loserRating - winnerRating) / 400));
  return Math.round(K * (1 - expected));
};
```

Deep explanation - what this code is responsible for

- calcEloDelta calculates a simple Elo-style rating gain for the winner.

- K=32 controls how much ratings can move.

- A lower-rated upset gives more points than an expected win.

Execution flow

- Compute expected winner score.

- Return rounded K * (1 - expected).

Edge cases / things interviewer may probe

- Only winner delta is computed; loser loses same amount elsewhere.

- This is custom, not official Codeforces rating.

How to improve in production

- Add draws/timeouts if needed.

- Store rating history for audit.

Possible interview questions and answers

Question	Answer
What does K mean?	It controls rating volatility.
Why use expected score?	Elo rewards upsets more and expected wins less.
Is this Codeforces rating?	No, it is internal CP Duel rating.
Can rating go negative?	Loser rating is floored at zero in handleWinner.

handleWinner()

```
const handleWinner = async (roomId, { player, subTime }, io) => {
  try {
    const room = await Room.findOne({ roomId, status: "ongoing" });
    if (!room) return;

    // Identify winner and loser players from room
    const winnerPlayer = room.players.find(
      (p) => String(p.user) === String(player.userId)
    );
    const loserPlayer = room.players.find(
      (p) => String(p.user) !== String(player.userId)
    );

    const winnerRating = winnerPlayer?.rating || 1200;
    const loserRating = loserPlayer?.rating || 1200;
    const delta = calcEloDelta(winnerRating, loserRating);

    // Update room doc
    // Update room doc
    await Room.findOneAndUpdate(
      { roomId, status: "ongoing" },
      {
        status: "finished",
        winner: player.userId,
        winnerUsername: player.username,
        winnerSubmissionTime: subTime,
        finishedAt: new Date(),
        ratingDelta: delta, // add this
        winnerNewRating: winnerRating + delta, // add this
        loserNewRating: Math.max(0, loserRating - delta), // add this
      }
    );

    // Update winner: wins++, rating += delta
    await User.findByIdAndUpdate(player.userId, {
      $inc: { wins: 1, rating: delta },
    });

    // Update loser: losses++, rating -= delta (floor at 0)
    if (loserPlayer) {
      const newLoserRating = Math.max(0, loserRating - delta);
      await User.findByIdAndUpdate(loserPlayer.user, {
        $inc: { losses: 1 },
        $set: { rating: newLoserRating },
      });
    }

    // Emit result with delta info
    io.to(roomId).emit("gameResult", {
      winner: player.username,
      winnerHandle: player.codeforcesHandle,
      submissionTime: subTime,
      problem: room.currentProblem,
      ratingDelta: delta, // winner gains this
      winnerNewRating: winnerRating + delta,
      loserNewRating: Math.max(0, loserRating - delta),
    });

    console.log(`Winner: ${player.username} | Delta +${delta}`);
  } catch (err) {
    console.error("handleWinner error:", err.message);
  }
};
```

Deep explanation - what this code is responsible for

- handleWinner finalizes the duel after a valid accepted submission is found.

- It updates the Room result, increments winner stats/rating, updates loser stats/rating, and emits gameResult.

- It queries the room with status ongoing to avoid finishing an already-finished duel.

Execution flow

- Find ongoing room.

- Identify winner/loser player snapshots.

- Read ratings.

- Calculate delta.

- Update Room to finished.

- Update winner User.

- Update loser User.

- Emit gameResult.

Edge cases / things interviewer may probe

- Room update and user updates are separate operations, so partial failure can cause inconsistency.

- If io is null, io.to would fail.

- Duplicate comment exists in source.

- Tie handling depends on startPolling winner choice.

How to improve in production

- Use MongoDB transactions.

- Guard io before emitting.

- Compare earliest submission time before calling handleWinner.

- Add tests for winner/loser rating updates.

Possible interview questions and answers

Question	Answer
Why query status ongoing?	It prevents changing a room already finished by another poll/update.
What DB documents change?	Room plus winner User plus loser User.
Why store winnerNewRating and loserNewRating in Room?	So history/result pages can show match-time rating outcome.
What is the consistency risk?	Separate DB updates can partially succeed; transactions would help.
What socket event is emitted?	gameResult with winner, problem, submission time, and rating details.

src/sockets/index.js

socketRoomMap

```
const socketRoomMap = new Map();
```

Deep explanation - what this code is responsible for

- socketRoomMap maps socket.id to roomId for connection cleanup.

- It is only live connection state; actual room state is in MongoDB.

Execution flow

- Set on createRoom/joinRoom.

- Delete on leaveRoom/disconnect.

Edge cases / things interviewer may probe

- It stores one room per socket. Multiple room membership would need a different structure.

How to improve in production

- Use socket.data.roomId or a richer connection registry.

Possible interview questions and answers

Question	Answer
Why not store room state here?	Socket memory is temporary; MongoDB is durable.
What happens on disconnect?	The map entry is removed.
Does this remove the player from MongoDB?	No, it only cleans live socket tracking.

initSocket()

```
const initSocket = (server) => {
  const io = new Server(server, {
    cors: {
      origin: [process.env.CLIENT_URL, 'http://localhost:5173', 'http://localhost:3000'],
      methods: ['GET', 'POST'],
      credentials: true,
    },
    // Important for Render (WebSocket behind proxy)
    transports: ['websocket', 'polling'],
  });

  // Give the duelController a reference to io
  setIO(io);

  io.on('connection', (socket) => {
    console.log(`Socket connected: ${socket.id}`);

    /**
     * createRoom event
     * Client emits after creating room via REST API
     * { roomId, username }
     */
    socket.on('createRoom', ({ roomId, username }) => {
      socket.join(roomId);
      socketRoomMap.set(socket.id, roomId);
      console.log(`${username} created and joined socket room ${roomId}`);
    });

    /**
     * joinRoom event
     * { roomId, username, players } -- players array from updated Room doc
     */
    socket.on('joinRoom', ({ roomId, username, players }) => {
      socket.join(roomId);
      socketRoomMap.set(socket.id, roomId);
      console.log(`${username} joined socket room ${roomId}`);

      // Notify everyone in the room that a new player joined
      io.to(roomId).emit('playerJoined', { players, roomId });
    });

    /**
     * startGame event
     * Host triggers this; actual logic runs through REST /api/duel/start
     * This event is informational -- confirms the start request was sent
     */
    socket.on('startGame', ({ roomId }) => {
      io.to(roomId).emit('gameStarting', { roomId });
    });

    /**
     * leaveRoom event
     * Player explicitly leaves
     */
    socket.on('leaveRoom', ({ roomId, username }) => {
      socket.leave(roomId);
      socketRoomMap.delete(socket.id);
      io.to(roomId).emit('playerLeft', { username });
    });

    /**
     * On disconnect -- clean up room membership
     */
    socket.on('disconnect', () => {
      const roomId = socketRoomMap.get(socket.id);
      if (roomId) {
        socketRoomMap.delete(socket.id);
        // If nobody left in socket room, room is effectively empty
        // Polling will eventually timeout or the room stays in DB
      }
      console.log(`Socket disconnected: ${socket.id}`);
    });
  });

  return io;
};
```

Deep explanation - what this code is responsible for

- initSocket creates and configures the Socket.IO server.

- It supports both websocket and polling transports, useful behind hosting proxies.

- It registers events that synchronize lobby/game state across connected clients.

- It calls setIO(io) so duelController can emit socket events after REST actions.

Execution flow

- Create Server.

- Configure CORS/transports.

- Call setIO.

- On connection, register createRoom/joinRoom/startGame/leaveRoom/disconnect handlers.

- Return io.

Edge cases / things interviewer may probe

- stopPolling is imported but unused.

- startGame is informational; actual start is REST.

- disconnect does not stop polling or update DB room state.

How to improve in production

- Remove unused import.

- Authenticate sockets with JWT.

- Use Redis adapter for multi-instance deployment.

- Handle reconnects and room rejoin more formally.

Possible interview questions and answers

Question	Answer
Why use Socket.IO rooms?	They let the server broadcast only to players in a given duel room.
Why is startGame informational?	The real state change happens in POST /api/duel/start; sockets only notify.
Why include polling transport?	Some deployments/proxies do not support pure WebSocket reliably.
What security improvement is needed?	Authenticate socket connections with JWT.
Does Socket.IO replace REST?	No. REST mutates durable state; sockets broadcast live updates.